

Document Scanner + OCR Pipeline

A Computer Vision Project Report

Course: CSE3010 – Computer Vision

Assignment: Build Your Own Project (VITyarthi)

Institution: VIT Bhopal University, School of Computing Science and Engineering

Submitted by: Sharanya

[Registration Number: _____]

1. Introduction

Photographs of physical documents taken with a phone camera are almost never perfectly aligned: they are skewed, perspective-distorted, and unevenly lit. This project builds a command-line pipeline that takes such a photo and produces a clean, flattened, high-contrast scan, then extracts the document's text via Optical Character Recognition (OCR). The pipeline is organized into three independent, testable modules that map directly onto the image-processing and geometric-transform concepts covered in the CSE3010 syllabus: image formation and enhancement, homography-based perspective correction, and feature/contour-based boundary detection.

2. Problem Statement

There is a need for a lightweight, GUI-free tool that converts a raw photo of a document into (a) a flattened, readable scan and (b) accurate, structured machine-readable text — without relying on a phone app or cloud service. This project addresses that need with a fully local, scriptable pipeline runnable from a terminal.

3. Functional Requirements

The system implements three major functional modules, as required by the assignment brief:

- **Module 1 – Document Detection & Perspective Correction:** locates the document boundary in the input photo (Canny edge detection + contour approximation) and applies a homography-based perspective warp to produce a top-down view.
- **Module 2 – Image Enhancement & Binarization:** converts the warped image to grayscale, applies CLAHE contrast equalization and denoising, then adaptive thresholding to produce a clean, scanner-quality black-and-white page.
- **Module 3 – OCR Text Extraction & Structured Output:** runs Tesseract OCR on the enhanced image and emits both plain text and a structured JSON file containing per-word text, confidence score, and bounding box.

Input/output structure: the CLI accepts a single image path (`--input`) and an output directory (`--output-dir`); it writes the intermediate warped image, the enhanced/binarized image, the extracted text file, and a structured JSON result.

Workflow: read image → detect & warp → enhance & binarize → OCR extract → write outputs → print summary. See the Process Flow diagram in Section 6.

4. Non-Functional Requirements

Requirement	How it is addressed
Performance	Image is resized before contour search (fixed 800px height) to bound processing cost; full pipeline runs in well under a second for typical inputs.
Reliability	A <code>DocumentNotFoundError</code> fallback path uses the full frame if no 4-point contour is found, so the pipeline degrades gracefully instead of crashing.
Usability	Single CLI entry point with argparse, clear <code>--help</code> text, sensible defaults, and a printed run summary (word count, confidence, elapsed time).
Maintainability	Each module (detector, enhancer, OCR) is a separate file with a narrow, documented public API, so any module can be modified or swapped independently.
Error handling	Every module validates its inputs (empty/invalid images) and raises specific exceptions; <code>main.py</code> catches and logs these without a raw stack trace reaching the user.

Logging / monitoring	Centralized logging (<code>utils.setup_logging</code>) writes timestamped, leveled logs to both the console and <code>scanner.log</code> .
Resource efficiency	Uses OpenCV's headless build (no GUI dependencies) and releases all intermediate arrays automatically via Python's garbage collector; no persistent state between runs.

5. System Architecture

The CLI (`main.py`) orchestrates three independent modules, each with a single responsibility, plus a shared utilities module for logging and I/O helpers.

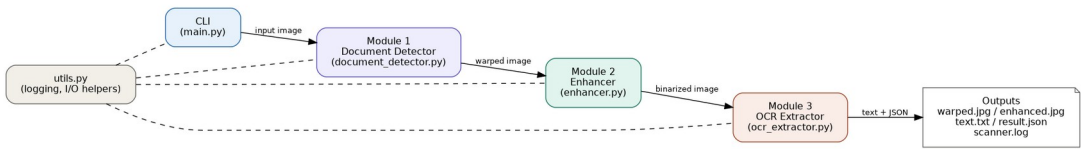


Figure 1: System architecture — module responsibilities and data handed between them.

6. Design Diagrams

6.1 Process Flow / Workflow Diagram

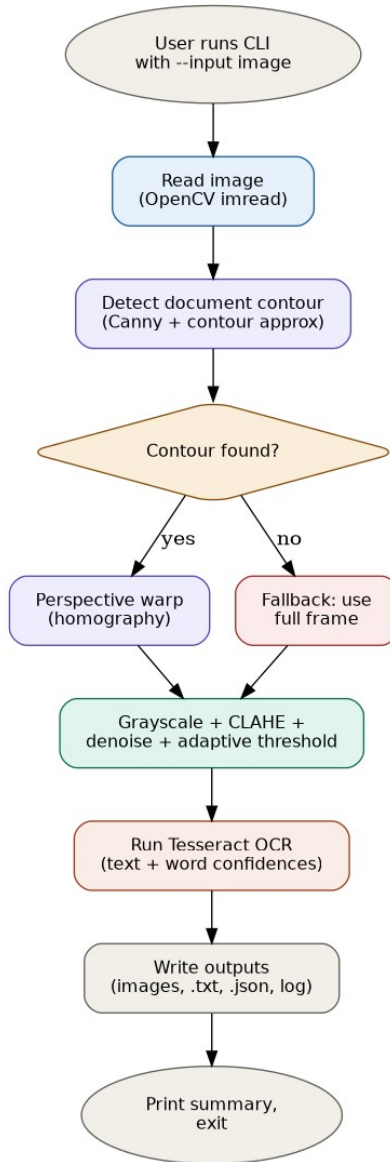


Figure 2: End-to-end processing workflow, including the contour-not-found fallback path.

6.2 Use Case Diagram

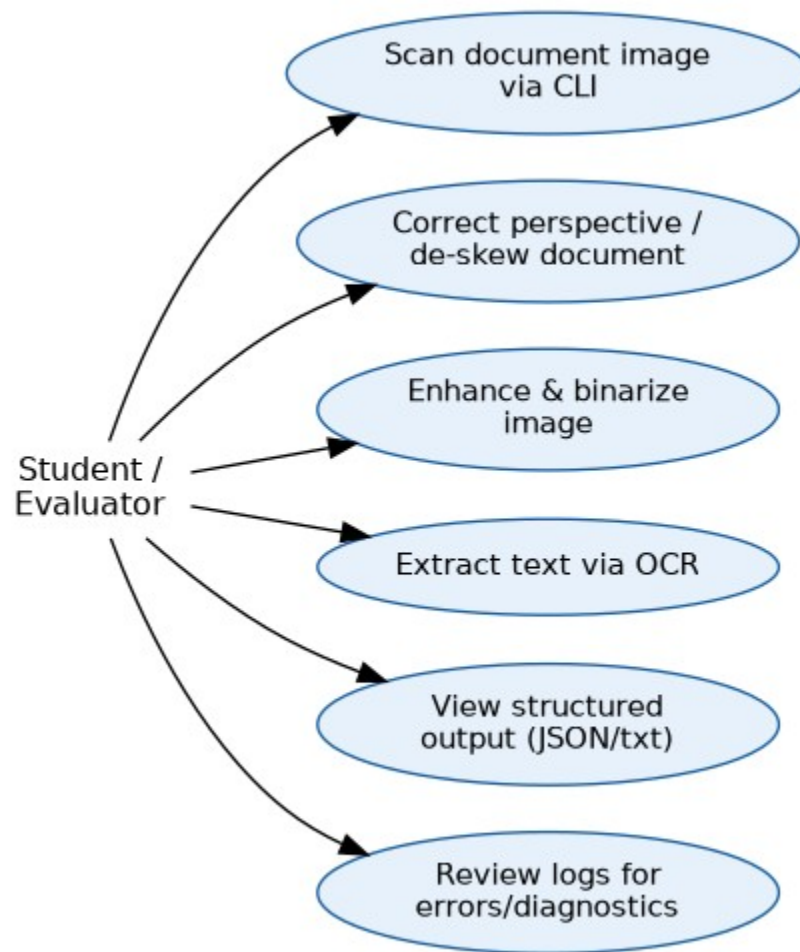


Figure 3: Use cases available to the student/evaluator running the CLI.

6.3 Class / Component Diagram

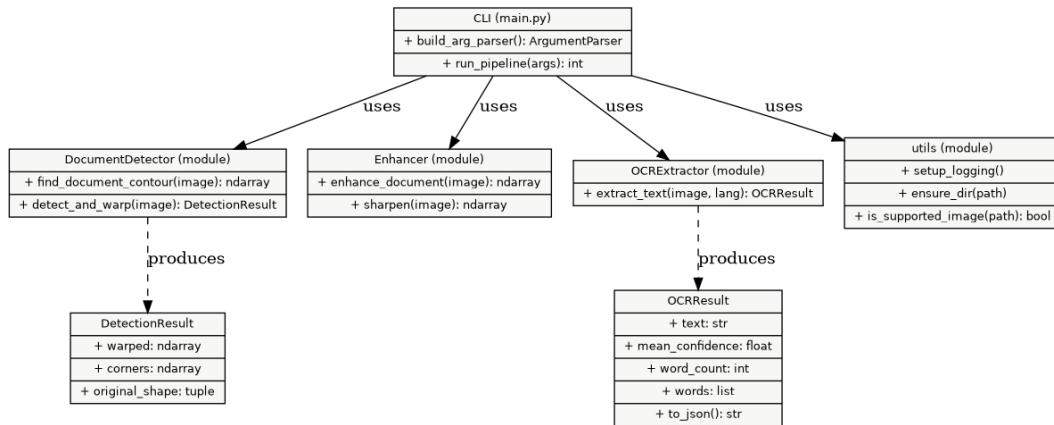


Figure 4: Module-level components, their public methods, and the data classes they produce.

6.4 Sequence Diagram

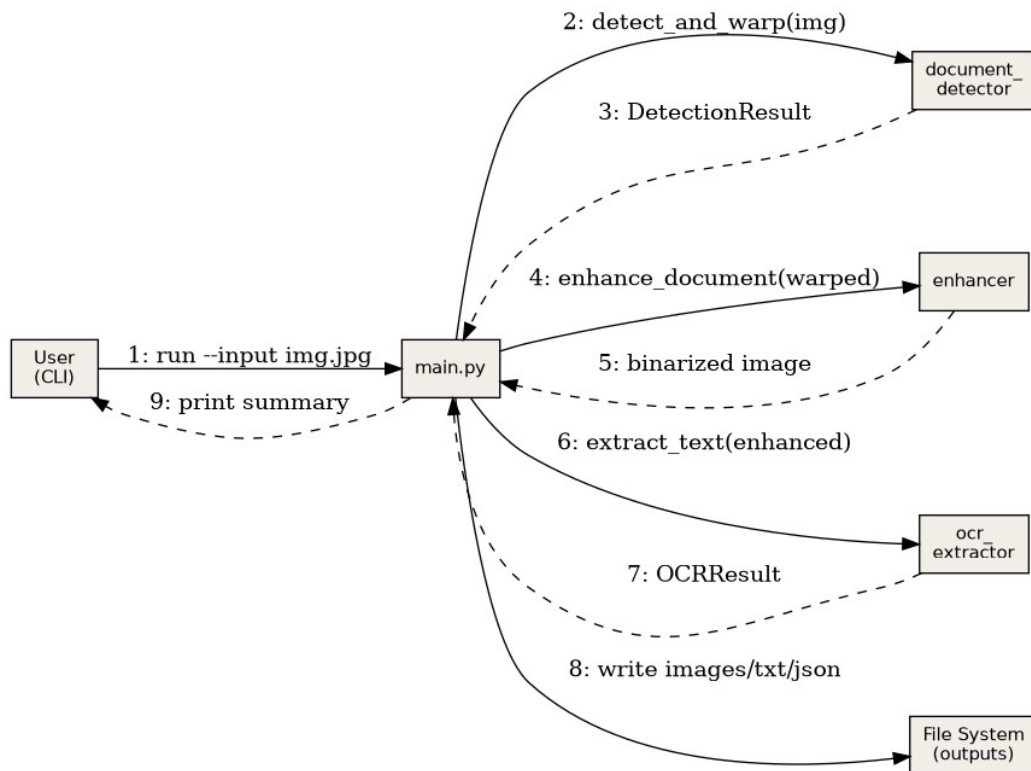


Figure 5: Call sequence for a single CLI invocation, from input image to written outputs.

6.5 Database / Storage Design

Not applicable — this project is a stateless, file-in/file-out CLI pipeline and does not use a database. Outputs are written directly to the filesystem as image, text, and JSON files (see Section 3).

7. Design Decisions & Rationale

- Three separate modules instead of one script: keeps each concept (geometry, image processing, OCR) independently testable and replaceable — e.g. Module 3 could be swapped for a cloud OCR API without touching Modules 1–2.
- Fallback-to-full-frame on failed detection: chosen over hard-failing so the pipeline is robust to photos where the document fills the whole frame or has low-contrast edges.
- Adaptive thresholding over a global threshold: documents photographed under uneven lighting have local brightness gradients that a single global threshold cannot handle well.
- JSON structured output alongside plain text: enables downstream automation (e.g. filtering by confidence, locating specific words) rather than only a human-readable transcript.
- CLI-only (argparse), no GUI: satisfies the assignment's command-line-executable requirement and keeps the tool scriptable/automatable.

8. Implementation Details

Language & libraries: Python 3, OpenCV (opencv-python-headless) for all image processing and geometric transforms, NumPy for array operations, and pytesseract as a thin wrapper around the Tesseract OCR engine.

8.1 Module 1 — `document_detector.py`

Resizes the input for fast contour search, applies Gaussian blur + Canny edge detection, finds the largest contours, and keeps the first one that approximates to a 4-point polygon (`cv2.approxPolyDP`). Corners are ordered (top-left, top-right, bottom-right, bottom-left) and a perspective transform matrix is computed with `cv2.getPerspectiveTransform`, then applied with `cv2.warpPerspective`.

8.2 Module 2 — `enhancer.py`

Converts to grayscale, optionally applies CLAHE (contrast-limited adaptive histogram equalization) to correct uneven lighting, denoises with `cv2.fastNlMeansDenoising`, and binarizes with `cv2.adaptiveThreshold` (Gaussian-weighted, block size 25).

8.3 Module 3 — `ocr_extractor.py`

Calls `pytesseract.image_to_string` for the plain-text transcript and `pytesseract.image_to_data` for per-word bounding boxes and confidence scores, aggregating them into an `OCRResult` dataclass with a `to_json()` serializer.

8.4 CLI — `main.py`

Wires the three modules together behind an `argparse` interface, writes all intermediate and final artifacts to the chosen output directory, and prints a run summary.

9. Screenshots / Results

The pipeline was validated on a synthetically generated test document (a skewed page with printed text on a dark background, generated in `tests/test_pipeline.py`) to provide a deterministic, reproducible demonstration.

HELLO WORLD

Computer Vision Test

Figure 6: Module 1 output — perspective-corrected, de-skewed document.

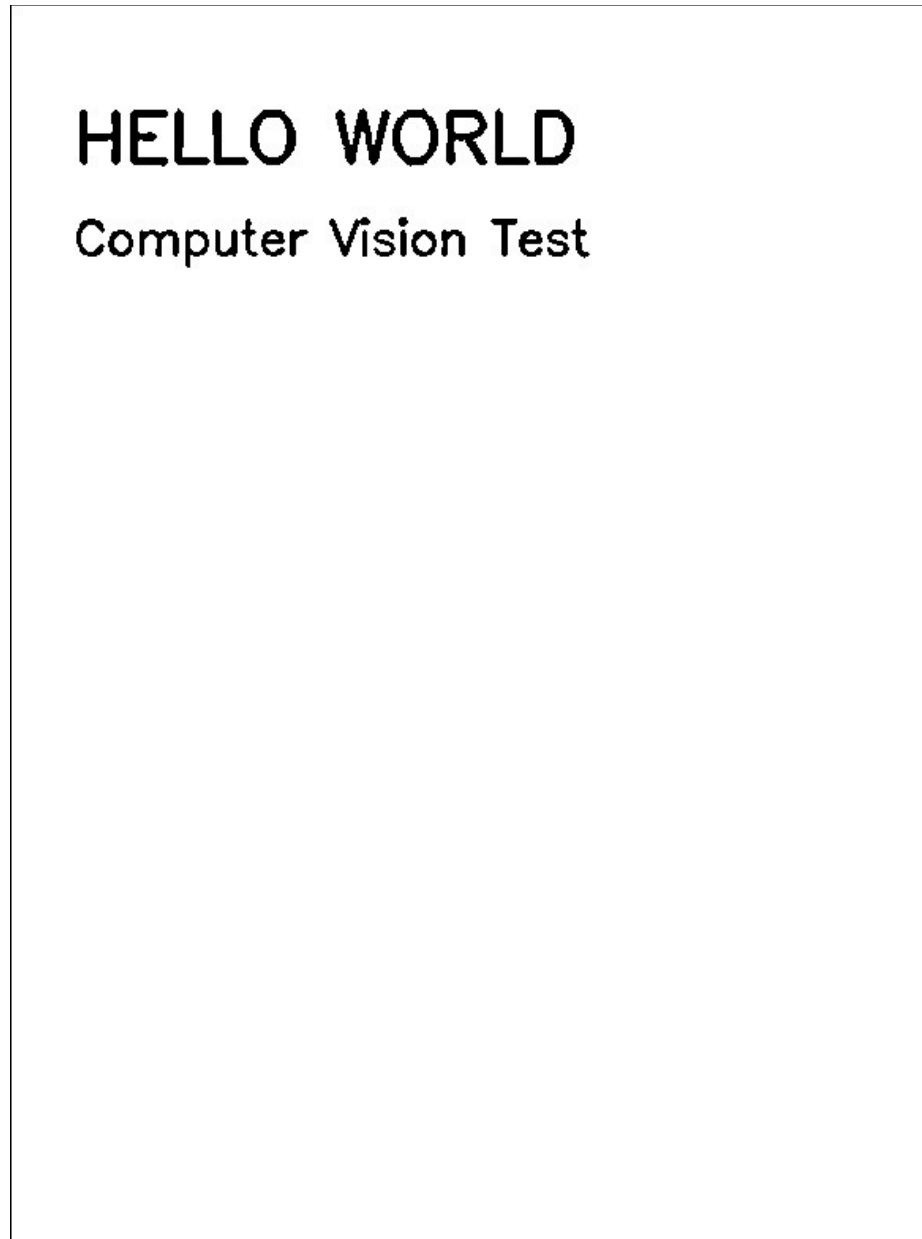


Figure 7: Module 2 output — enhanced, binarized scan ready for OCR.

Module 3 (OCR) output on this sample:

```
{  "text": "HELLO WORLD\n\nComputer Vision Test",  "mean_confidence": 96.0,
"word_count": 5,  "words": [ { "text": "HELLO", "confidence": 96.0, "bbox":
{...} }, ... ] }
```

Run summary printed by the CLI:

```
Words extracted:      5 Mean OCR confidence: 96.0% Time elapsed:      0.59s
```

10. Testing Approach

Automated unit tests (tests/test_pipeline.py, run via pytest) cover all three modules:

- Document Detector: verifies a valid `DetectionResult` is returned for a normal image, that an empty image raises `DocumentNotFoundError`, and that a blank/featureless image correctly falls back to using the full frame.
- Enhancer: verifies output is strictly binary (0/255 pixel values only) and that an empty image raises `ValueError`.
- OCR Extractor: verifies text is extracted from a synthetic page, `mean_confidence` is a float, an empty image raises `ValueError`, and JSON serialization includes expected fields.

All 8 tests pass. Synthetic test images are generated on the fly (no external fixture files needed), keeping the test suite fully self-contained and deterministic. The pipeline was additionally validated end-to-end via the CLI itself (Section 9).

11. Challenges Faced

- Robust corner ordering: perspective warps break if the four detected corners aren't consistently ordered (TL/TR/BR/BL); solved using the sum/difference-of-coordinates trick.
- Handling failed contour detection gracefully: rather than letting the pipeline crash on low-contrast or edge-filling images, a full-frame fallback path was added and unit-tested explicitly.
- Uneven lighting in photographed documents: a plain global threshold produced patchy results; switching to adaptive Gaussian thresholding (with CLAHE beforehand) fixed this.
- Deterministic testing without real photos: building a synthetic-document generator (skewed page + printed text) so tests don't depend on external image fixtures or manual photography.

12. Learnings & Key Takeaways

- Practical experience applying homography/perspective-transform theory (Module 2 of the syllabus) to a real image, not just synthetic point sets.
- Adaptive, locally-aware image processing (CLAHE, adaptive thresholding) matters far more than global operations once inputs come from uncontrolled, real-world conditions like phone photography.
- Designing for graceful degradation (fallback paths, input validation) is as important as the 'happy path' algorithm for a tool meant to be run by someone else.
- A clean module boundary (detector → enhancer → OCR) made independent unit testing straightforward and would make it easy to later swap any one stage (e.g. a different OCR engine) without touching the rest.

13. Future Enhancements

- Multi-page / batch processing: accept a directory of images or a PDF and process all pages in one run.
- Automatic rotation correction: detect and correct 90°/180° rotated documents in addition to perspective skew.
- Layout-aware OCR: group extracted words into lines/paragraphs/table cells instead of a flat word list.
- Configurable OCR backend: abstract Module 3 behind an interface so a cloud OCR API can be swapped in for higher accuracy on handwriting.

14. References

- R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer-Verlag London Limited, 2011.
- R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd Edition, Cambridge University Press, 2004.
- OpenCV Documentation: <https://docs.opencv.org/>

- Tesseract OCR / pytesseract Documentation: <https://github.com/tesseract-ocr/tesseract> ,
<https://github.com/madmaze/pytesseract>
- CSE3010 Computer Vision course syllabus, VIT Bhopal.